

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 60%

Dr Ramamoorthy

QUESTIONS: 12

Total Marks: 60

Annexure A

CONCEPT MAPPING

Code of Conduct:

I RASHMI NAURENE [192521357] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

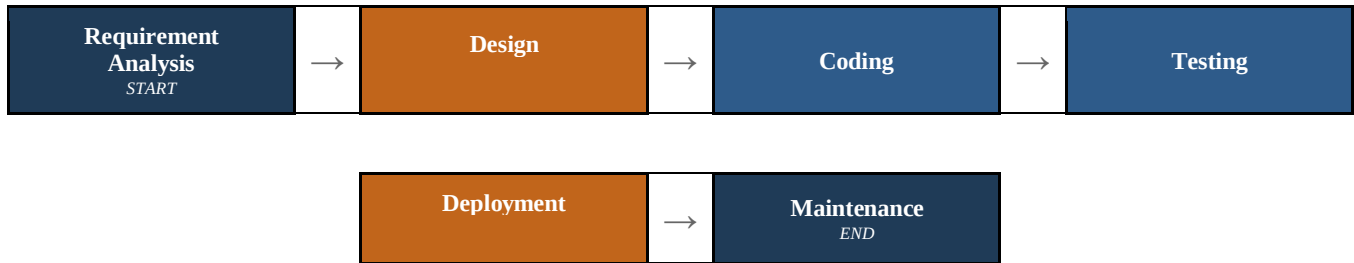
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Construct a Concept Map

Requirement Analysis → Design → Coding → Testing → Deployment → Maintenance

Concept Map



Explanation

- The Design stage sits between analysis and coding because a system cannot be built directly from a requirements list — it needs an architectural blueprint (modules, data flow, interfaces) that programmers can actually follow.
- Deployment is the handover point where a tested build stops being an internal artifact and becomes a live system that real users can access, which is why it comes after testing and before ongoing maintenance.
- Placed together, the six stages form a closed loop: what is learned during maintenance typically feeds back into the next round of requirement analysis.

Analysis

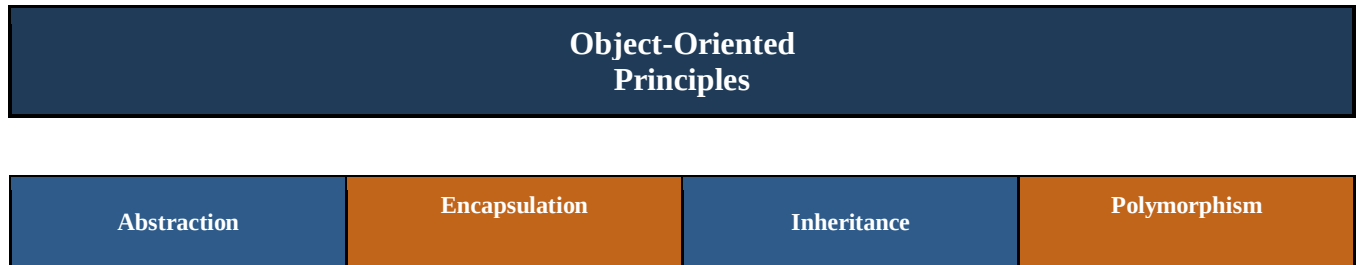
- Each stage acts as a checkpoint, so an error missed in one phase (say, an ambiguous requirement) usually costs far more to fix once it reaches coding or testing.
- Because the phases are sequential, the model gives a project manager a clear way to track progress and allocate time or budget per stage.
- Skipping or rushing the Design or Deployment stage is a common source of avoidable rework later in the project, which is why both are treated as mandatory rather than optional steps.

(5 Marks)

2. Construct a Concept Map

Abstraction → Encapsulation → Inheritance → Polymorphism

Concept Map



Explanation

- Encapsulation is the mechanism that bundles an object's data with the methods that act on it, and restricts outside code from reaching in and changing that data directly — it is the 'lock' that abstraction's 'interface' sits on top of.
- Polymorphism lets a single method name behave differently depending on which object it is called on, which is what allows a program to treat many related object types through one common set of operations.
- The four principles are not independent add-ons; abstraction defines what an object exposes, encapsulation protects how it does so internally, inheritance shares that design across related classes, and polymorphism lets those related classes respond differently to the same call.

Analysis

- Because encapsulation hides internal state, other parts of the program cannot corrupt an object by mistake — this is what people mean when they say OOP improves data safety.
- Polymorphism reduces the amount of conditional (if/else) logic needed to handle different object types, which keeps the codebase shorter and easier to extend.
- Together, the four principles let a development team change or add classes without rewriting the code that already depends on them, which is the real source of OOP's long-term maintainability advantage.

(5 Marks)

3. Construct a Concept Map

Concept Map

Class	Blueprint
Object	Instance
Method	Behavior
Attribute	State

Explanation

- A method is where an object's behaviour lives — it is the code that runs when the object is asked to do something, such as calculate a total or send a notification.
- An attribute is where an object's state lives — the current values that distinguish one object from another of the same class, for example one Student object having a different GPA from another.
- The pairing pattern here mirrors a simple rule: a class is a design-time template, while an object is what exists while the program is actually running, and methods/attributes are the two things every object carries with it.

Analysis

- Separating behaviour (methods) from state (attributes) makes it possible to change how something is computed without touching the data it operates on, or vice versa.
- This distinction is what makes objects easy to reason about individually — you can inspect an object's current state and predict its behaviour without reading the rest of the program.
- It also gives a natural way to organise a class definition: attributes describe 'what the object has,' methods describe 'what the object can do.'

(5 Marks)

4. A Concept Map Shows

Concept Map



Explanation

- Usability is the quality attribute that captures how quickly and comfortably a real user can accomplish a task with the software, independent of whether the underlying code is well written.
- It sits alongside reliability and maintainability as one of three lenses on 'quality' — reliability judges the software from the system's point of view (does it keep working correctly), while usability judges it from the user's point of view.
- None of the three attributes is sufficient alone: a program can be perfectly reliable and still fail commercially if it is unusable, or easy to use but unreliable enough that users stop trusting it.

Analysis

- Usability problems are often the first thing users notice, so they have an outsized effect on adoption even when the underlying logic is technically sound.
- Because usability, reliability, and maintainability pull in different directions during development (extra validation improves reliability but can slow the interface down), trade-offs between them have to be made deliberately rather than by accident.
- Tracking all three together gives a more complete quality picture than optimising for just one, which is why they are grouped under the same parent concept here.

(5 Marks)

5. UML Concept Map

Concept Map

UML	
Structural Diagrams	Object Diagram
Behavioral Diagrams	Activity Diagram
Interaction Diagrams	Communication Diagram

Explanation

- An Object Diagram is the structural-diagram example here because it captures a snapshot of actual object instances and the links between them at one moment in time, rather than the general class-level structure a class diagram would show.
- A Communication Diagram belongs under Interaction Diagrams because, like a sequence diagram, its purpose is to show how objects exchange messages — but it emphasises the network of relationships between objects rather than strict time ordering.
- Grouping diagrams this way reflects what question each type answers: structural diagrams answer 'what exists,' behavioral diagrams answer 'what happens over time,' and interaction diagrams answer 'who talks to whom.'

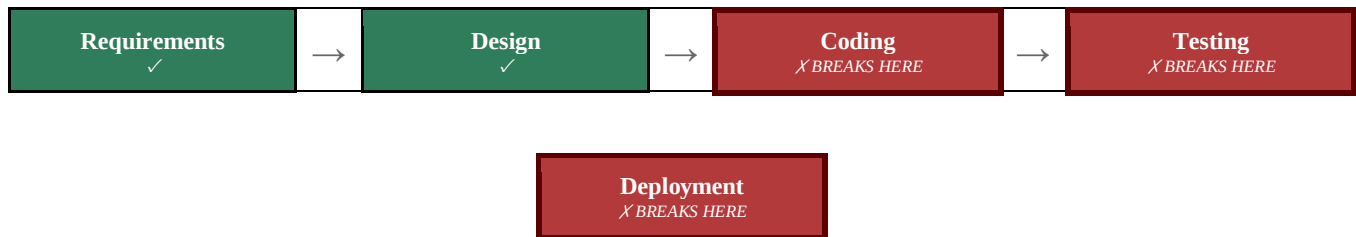
Analysis

- Picking the right diagram type for the right question avoids the common mistake of trying to force one diagram (usually the class diagram) to explain everything about a system.
- Object and communication diagrams are especially useful for debugging or explaining a specific runtime scenario, since they show concrete instances rather than abstract templates.
- Having a structured taxonomy like this also makes it easier for a documentation set to stay consistent — each new diagram added to a project has an obvious category to belong to.

(5 Marks)

6. SDLC Failure Point Analysis

Concept Map



Explanation

- The chain shows Requirements and Design completing cleanly, which rules out a misunderstood problem statement or a flawed architecture as the root cause.
- The break appears at Coding, meaning the design was sound but its translation into working program logic introduced defects — this is typically a bug in conditionals, data handling, or an incomplete implementation of a designed component.
- Because Testing and Deployment are both marked as failed downstream of Coding, they are consequences of the coding defect rather than separate, independent problems.

Analysis

- A defect introduced at Coding blocks Testing from giving a meaningful signal, since tests will keep failing for the same underlying reason until the code itself is fixed.
- Left uncorrected, this kind of failure propagates forward — Deployment is effectively frozen until the coding fault is resolved, which stalls the whole project timeline, not just one phase.
- The earlier the break is caught relative to Deployment, the cheaper it is to fix, which is the main argument for catching coding defects through review rather than waiting for end-to-end testing.

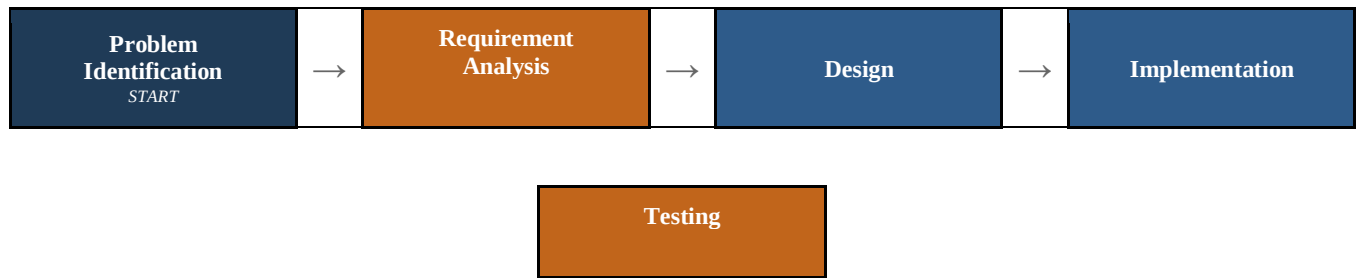
Solution

- Trace the failing test cases back to the specific function or module responsible, rather than re-testing the whole system blindly.
- Have a second developer review the flagged code section against the original design document to catch logic that technically runs but doesn't match intent.
- Add or update unit tests around the fixed section so the same defect is caught automatically if it resurfaces later.

(5 Marks)

7. Construct a Concept Map

Concept Map



Explanation

- Requirement Analysis has to follow Problem Identification because naming a problem is not the same as specifying what a solution needs to do — this step turns a general problem statement into concrete, checkable requirements.
- Testing closes the chain because it is the only stage that formally checks whether Implementation actually satisfies what Requirement Analysis specified earlier — without it, there is no evidence the two stages ever matched.
- The five stages together describe a single feedback loop: what Testing finds often sends the process back to an earlier stage rather than simply ending it.

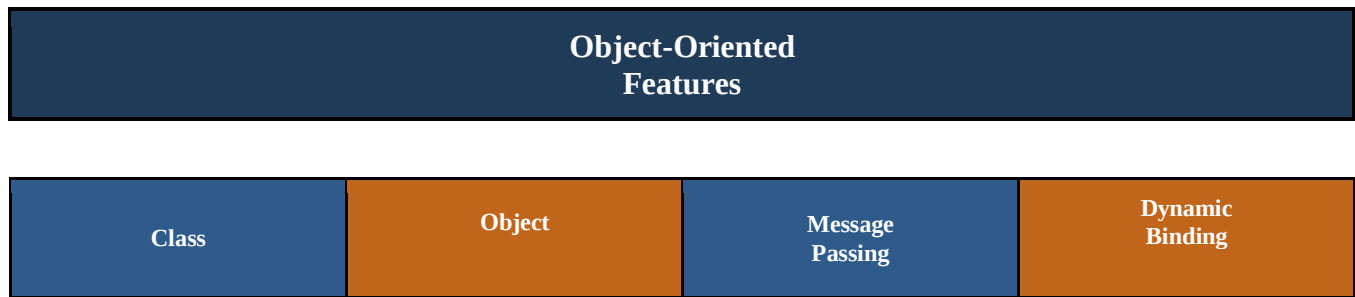
Analysis

- Because Testing is defined against the original requirements rather than against the code itself, this structure prevents a subtle failure mode where an implementation is technically bug-free but still solves the wrong problem.
- Requirement Analysis is the cheapest place to catch a misunderstanding, so investing time there reduces the amount of rework needed once Design and Implementation are underway.
- This chain works as a lightweight audit trail — for any finished feature, you can trace it back through Implementation and Design to the specific requirement that justified building it.

(5 Marks)

8. Construct a Concept Map

Concept Map



Explanation

- An Object is what a Class becomes once the program actually runs — the class is a static description, and the object is the live thing in memory that holds real values and can be acted on.
- Dynamic Binding is the rule that decides, at the moment a message is sent, exactly which version of a method should actually execute — this matters most when a subclass has overridden a method inherited from its parent.
- Message Passing is the glue between the two: objects don't call each other's code directly, they send messages, and dynamic binding is what resolves each message to the correct method at runtime.

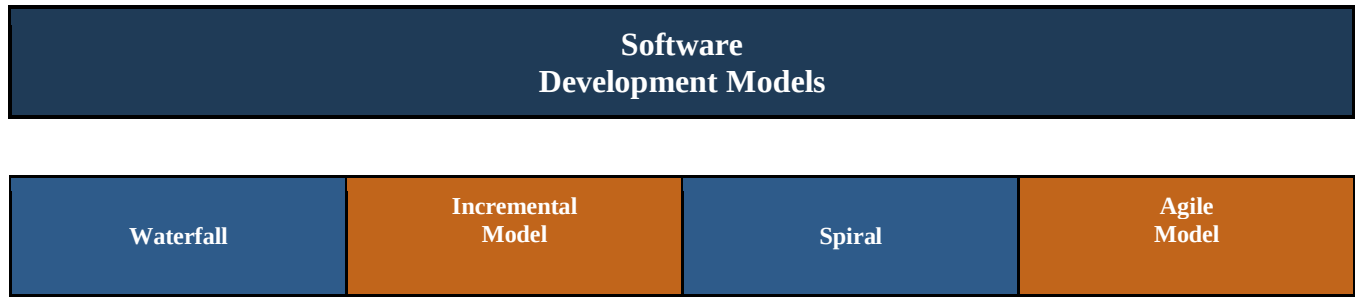
Analysis

- Without dynamic binding, message passing would always call the same fixed method regardless of the object's actual subclass, which would defeat the purpose of having subclasses override behaviour in the first place.
- This runtime resolution is what makes it possible to add a brand-new subclass later and have existing code that sends messages to it work correctly without modification.
- The four concepts form a dependency chain rather than a flat list: objects come from classes, messages are how objects interact, and dynamic binding is what makes that interaction flexible.

(5 Marks)

9. Construct a Concept Map

Concept Map



Explanation

- The Incremental Model breaks a project into a series of smaller builds, each adding new functionality to what was delivered before, rather than requiring the whole system to be finished before anything is released.
- The Agile Model formalises iteration further by wrapping short development cycles around continuous conversation with the customer, so requirements can be adjusted between cycles instead of being locked in at the start.
- Placed next to Waterfall and Spiral, these four models represent a spectrum from rigid and sequential (Waterfall) to flexible and customer-driven (Agile), with Incremental and Spiral occupying the middle ground in different ways.

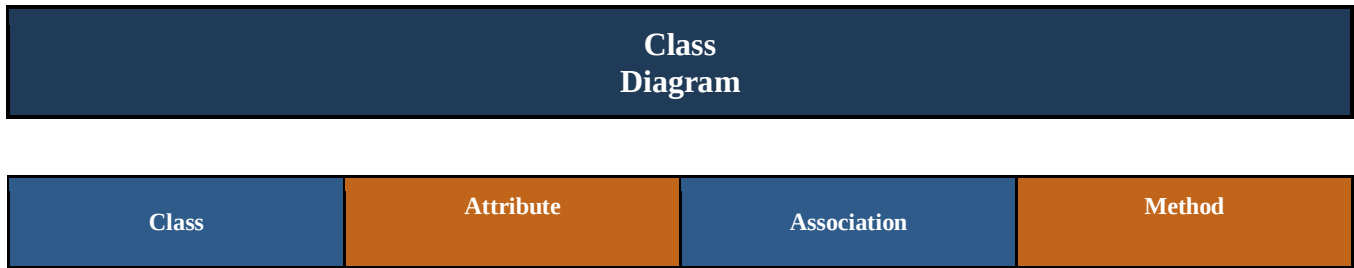
Analysis

- Incremental delivery gives stakeholders something usable earlier in the project, which makes it easier to catch a wrong assumption before the entire budget has been spent.
- Agile's tight feedback loop is particularly suited to projects where the customer cannot fully specify requirements up front, which is common in consumer-facing software.
- None of the four models is universally 'best' — the right choice depends on how well the requirements are understood at the start and how much the client needs to be involved along the way.

(5 Marks)

10. Construct a Concept Map

Concept Map



Explanation

- An Attribute is one of the two things a class box in a UML class diagram is required to show, representing a piece of data the class holds (for example, a Student class holding a rollNumber attribute).
- A Method is the other required element, representing an operation the class can perform, and it is typically listed directly beneath the attributes inside the same class box.
- Association is drawn as a separate element outside the class box — a line connecting two classes — and represents a relationship between them rather than something owned by a single class.

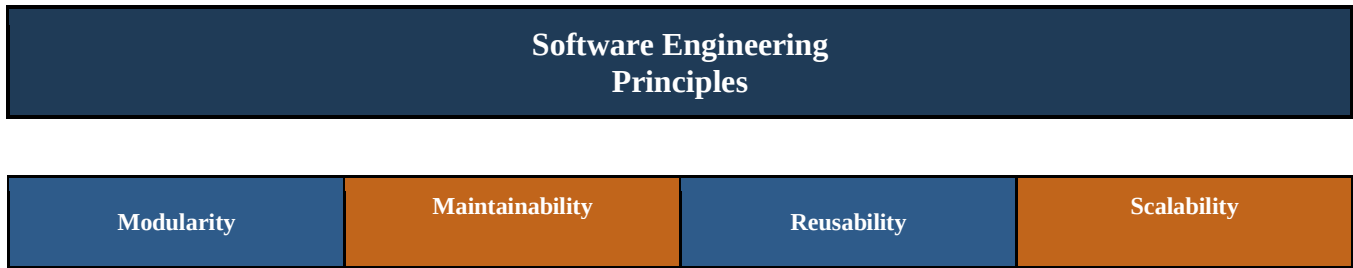
Analysis

- Showing attributes and methods together inside one box is what makes a class diagram self-contained: a reader can see both the data and the behaviour of a class without consulting other diagrams.
- Associations extend the diagram from describing individual classes to describing the system as a whole, since they reveal how classes are meant to interact with one another.
- Because attributes, methods, and associations are visually distinct elements, the diagram lets a developer separate 'what a class contains' from 'how it connects to others' at a glance.

(5 Marks)

11. Construct a Concept Map

Concept Map



Explanation

- Maintainability describes how easily a system can be corrected, updated, or extended after it is already in use, and it is listed right after Modularity because breaking a system into modules is one of the main ways maintainability is achieved in practice.
- Scalability describes a system's ability to keep performing acceptably as demand grows — more users, more data, or more transactions — without a complete redesign.
- The four principles reinforce each other: modular components are what make a system both easier to maintain and easier to reuse, and a well-structured, reusable codebase is generally easier to scale than a monolithic one.

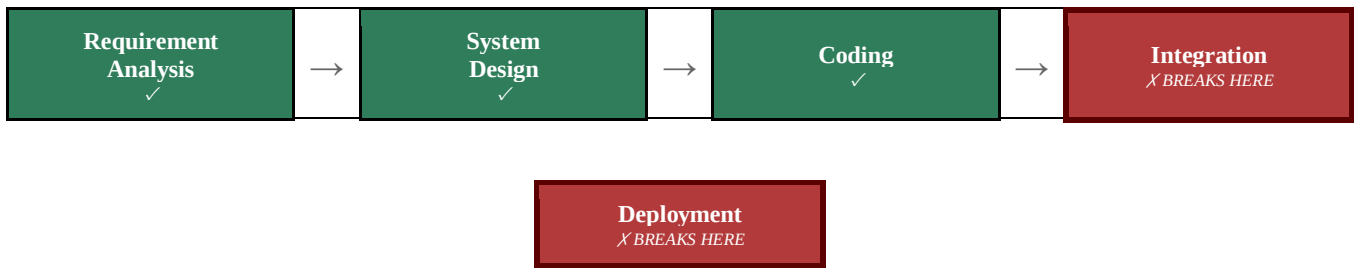
Analysis

- Modularity is best understood as the structural cause, with maintainability and reusability as two of its direct effects — this is why the three are often discussed together rather than as unrelated qualities.
- Scalability failures are frequently traced back to a lack of modularity earlier in the design, since tightly coupled components are hard to scale independently.
- Applying all four principles together tends to lower the long-term cost of a software project even if it adds some upfront design effort, which is the central trade-off software engineering asks teams to accept.

(5 Marks)

12. SDLC Failure Point Analysis

Concept Map



Explanation

- Unlike the earlier failure case, Coding itself completes successfully here — each module was presumably built and unit-tested against its own specification without issue.
- The break appears specifically at Integration, which is the stage where independently built modules are combined and expected to work together — this points to a mismatch in how modules were designed to interact, such as inconsistent data formats or incorrect assumptions about a shared interface.
- Deployment is marked failed only as a downstream consequence: a system that cannot be integrated correctly cannot be safely released, regardless of how solid any individual module is.

Analysis

- This failure pattern is a classic sign that modules were developed too independently, without enough shared agreement on interfaces during the Design stage.
- Integration failures are often harder to diagnose than coding failures, because the individual modules may each report no errors when tested alone.
- The gap between a successful Coding phase and a failed Integration phase highlights why interface contracts need to be agreed on before, not after, modules are built separately.

Solution

- Run integration tests specifically at module boundaries rather than relying only on each module's own unit tests.
- Confirm that every module agrees on the same data formats, naming conventions, and API contracts before combining them.
- Resolve any interface mismatches found during integration testing before the build is scheduled for deployment.

(5 Marks)

Rubrics for Calculation — Concept Mapping

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand